Shoehorning Apache Onto your Box: System Sizing Tips

By Sander Temme <sander@temme.net>

The Problem

You are responsible for a web server. There it sits, in a rack in the server room, or on a card table in the back of the office, next to that coffee machine that overflows every once in a while. Maybe you inherited this machine, or perhaps you bought it with your own hard-earned cash or inadequate IT budget. Now you are tasked with making that web server work and generate all-important revenue for the company. With the Christmas season just around the corner, everything depends on you.

The Premise

The following are the assumptions upon which this article is based:

- You own one machine, or a few, serving your own or your company's website.
- You are interested in getting the maximum performance from this existing infrastructure.
- There is, at this time, no budget to buy new hardware.
- Your machines are running Linux, FreeBSD or Solaris as an operating system.
- You did not compile your Apache web server, and do not wish to do so.

Fortunately, performance tuning under these conditions is not the daunting task it may seem!

Monitoring Your Server

The first task when sizing or performance-tuning your server is to find out how your system is currently performing. By monitoring your server under normal load, or under artificially generated load, you can extrapolate its behavior under stress.

top

The '*top'* tool ships with Linux and FreeBSD, and can be downloaded for Solaris[1]. It collects a bunch of statistics for the system and for each running process, then displays them interactively on your terminal. The data displayed is refreshed every second and varies by platform, but typically includes system load average, number of processes and their current states, the percent CPU(s) time spent executing user and system code, and the state of the virtual memory system. The data displayed for each process is typically configurable and includes its process name and ID, priority and *nice* values, memory footprint, and percentage CPU usage.

Top is a wonderful tool even though it's slightly resource intensive (when running, its own process is usuallyin the top ten CPU gluttons). It is indispensable in determining the size of a running process, which comes in handy in the paragraph *'Generating A Test Load'* described below.

---

[1] http://www.sunfreeware.com/

free

This command is only available on Linux. It shows how much memory and swap space is in use. Linux allocates unused memory as file system cache. The *free* command shows usage both with and without this cache. The *free* command can be used to find out how much memory the operating system is using, as described in the paragraph 'Sizing MaxClients' below. The output of *free* looks as follows:

```
[root@gaydeceiver root]# free
             total       used       free     shared    buffers     cached
Mem:        256064     248744       7320          0      64028      79560
-/+ buffers/cache:     105156     150908
Swap:       511992       6712     505280
```

vmstat

This command is available on all three platforms. It displays a large number of operating system metrics. Run without argument, it displays a status line for that moment. When a numeric argument is added, the status is redisplayed at designated intervals. For example, 'vmstat 5,' causes the information to reappear every five seconds. Vmstat displays the amount of virtual memory in use, how much memory is swapped in and out each second, the number of processes currently running and sleeping, the number of interrupts and context switches per second and the usage percentages of the CPU.

Web Server Logs

Apache creates log files, which can yield a lot of information about server performance. The *error_log* will contain messages if the server has reached the maximum number of active processes or the maximum number of concurrently open files. The error_log also reflects when processes are being spawned at a higher-than-usual rate in response to a sudden increase in load. Indeed, any error encountered by Apache will appear in this log, so it is good practice to review it frequently.

SE Toolkit

The SE Toolkit by Rich Pettit is a system monitoring toolkit for Solaris. Currently, its home page[2] seems to be offline, although a cached copy of the page on Google mentions the domain being renewed in August 2003, implying the product is still active and is likely to reappear for download soon. The SE Toolkit works with scripts based on the C preprocessor and comes with a number of sample scripts. This toolkit is discussed in the book *Sun Performance and Tuning*, listed in the *'Further Reading'* section below.

mod_status

The mod_status module gives an overview of the server performance at a given moment. It generates an HTML page with, among others, the number of Apache processes running and how many bytes each has served, and the CPU load caused by Apache and the rest of the system. The Apache Software Foundation uses mod_status on their own web site[3]. If you put the *ExtendedStatus On* directive in your httpd.conf, the mod_status page will

[2] http://www.setoolkit.com/

[3] http://www.apache.org/server-status

give you more information at the cost of a little extra work per request. (This sentence is vague).

Generating A Test Load

It is useful to generate a test load to monitor system performance under realistic operating circumstances. Besides a number of commercial packages such as *LoadRunner*, there are a number of freely available tools to generate a test load against your web server.

- Apache ships with a test program called *ab*, short for *Apache Bench*. It can generate a web server load by repeatedly asking for the same file in rapid succession. You can specify a number of concurrent connections and have the program run for either a given amount of time or a specified number of requests.
- Another freely available load generator is *http_load*[4]. This program works with a URL file and can be compiled with SSL support.
- The Apache Software Foundation offers a tool named *flood*[5]. *Flood* is a fairly sophisticated program that is configured through an XML file. Version 1.1 of *flood* was released recently, but since the project uses APR it can be kind of a challenge to compile.
- Finally, JMeter, a Jakarta subproject, is an all-Java load-testing tool. While early versions of this application were slow and difficult to use, the current version 1.9.1 seems to be versatile and useful.

When you load-test your web server, please keep in mind that, if that server is in production, the test load may negatively affect the server's response. Also, any data traffic you generate may be charged against your traffic allowance.

How Many Processes?

The Apache *httpd* is a pre-forking web server. When the server starts, the parent process spawns a number of child processes that do the actual work of servicing requests. If you're using a threaded Multi-Processing Module (MPM) with Apache 2.0, each child process will run a number of threads to service requests. The maximum number of *workers*, be they child processes or threads within a process, is a rough estimate of how many requests your server can serve concurrently. It is a merely a rough estimate because the kernel can queue connections attempts for your web server. When your site becomes busy and the maximum number of workers are running, the machine doesn't hit a hard limit beyond which clients will be denied access. However, once requests start backing up, system performance is likely to degrade.

MaxClients

The MaxClients directive in your Apache *httpd* configuration file[6] specifies the maximum number of workers your server can create. It has two cousins, the

---

[4] http://www.acme.com/software/http_load/

[5] http://httpd.apache.org/test/flood/

[6] This file is usually named *httpd.conf* and can be found in various places depending on your installation. If you installed Apache from source or a binary distribution, you can

MinSpareServers and *MaxSpareServers* directives, which specify the number of workers Apache keeps waiting in the wings ready to serve requests. The absolute maximum number of processes is hardcoded into Apache 1.3 as the parameter HARD_SERVER_LIMIT: in order to change it you'd have to recompile the server. Fortunately, most distributors have raised this limit well beyond the default of 256. In Apache 2.0, this limit is configurable through the *ServerLimit* directive.

Spinning Threads

For Apache 1.3, or the *prefork* MPM of Apache 2.0, that's all there is to determining the process limit. However, If you are running a threaded MPM the situation is a little more complicated. Threaded MPMs support the *ThreadsPerChild* directive[7]. Apache requires that *MaxClients* is evenly dividable by *ThreadsPerChild*. If you set either directive to a number that doesn't meet this requirement, Apache will complain to the error log and adjust the ThreadsPerChild value.

Sizing MaxClients

Optimally, the maximum number of processes should be set so that all the memory on your system is used, but no more. If your system gets so overloaded that it needs to heavily swap core memory out to disk, performance will degrade quickly.

The formula for determining *MaxClients* is fairly simple:

MaxClients = (total RAM – RAM for OS – RAM for external programs) / (RAM per *httpd* process)

The various amounts of memory allocated for the OS, external programs and the *httpd* processes is best determined by observation: use the *top* and *free* commands described above to determine the memory footprint of the OS without the web server running. You can also determine the footprint of a typical web server process from top: most top implementations have a *Resident Size* (RSS) column and a *Shared Memory* column. The difference of these two is the amount of memory per-process: the shared segment really exists only once and is used for the code and libraries loaded and the dynamic inter-process tally, or 'scoreboard,' that Apache keeps. How much memory each process takes for itself depends heavily on the number and kind of modules you use. The best approach here is to generate some test load against your web site and see how big the *httpd* processes get.

The 'RAM for external programs' parameter is intended mostly for CGI programs and scripts that run outside the web server process. But, if you have a Java virtual machine running Tomcat on the same box: that needs a significant amount of memory too.

The above assessment should give you an idea how far you can push *MaxClients*, but it is not an exact science. When in doubt, be conservative and use a low *MaxClients* value. The Linux kernel will put extra memory to good use for caching disk access, and on Solaris you need enough available real RAM memory to create any process. If no real

---

find it in the *conf* directory under your install directory. If Apache came pre-installed, you may find it somewhere under */etc,* or under */usr/local/etc*.

[7] http://httpd.apache.org/docs-2.0/mod/mpm_common.html#threadsperchild

memory is available, Apache will start writing 'No space left on device' messages to the error log.

Selecting your MPM

The prime reason for selecting a threaded MPM is that threads consume fewer system resources than processes, and it takes less effort for the system to switch between threads. This is more true for some operating systems more than others. On systems like Solaris and AIX, manipulating processes is relatively expensive in terms of system resources. On these systems, running a threaded MPM makes sense. On Linux, the threading implementation actually uses one process for each thread. Linux processes are relatively lightweight, but it means that a threaded MPM has less of a performance advantage than in other environments.

Running a threaded MPM can cause stability problems in some situations: should a child process of a preforked MPM crash, at most one client connection is affected. If a threaded child crashes, all the threads in that process disappear, which means all the clients currently being served by that process will see their connection aborted. Additionally, there may be so-called *thread-safety* issues, especially with third-party libraries. In threaded applications, threads may access the same variables indiscriminantly, not knowing whether a variable may have been changed by another thread.

This has been a sore point within the PHP community[8]. The PHP processor heavily relies on third-party libraries and cannot guarantee that all of these are thread-safe. The good news is that, if you are running Apache on Linux, you can run PHP in the preforked MPM without fear of losing too much performance relative to the threaded option.

Spinning Locks

Apache maintains an inter-process lock around its network listener. For all practical purposes, this means that only one *httpd* child process can receive a request at any given time. The other processes are either servicing requests already received or are '*camping out'* on the lock, waiting for the network listener to become available. This process is best visualized as a revolving door, with only one process allowed in the door at any time. On a heavily loaded web server with requests arriving constantly, the door spins quickly and requests are accepted at a steady rate. On a lightly loaded web server, the receiving process may have to stay in the door for a while, durinch which all the other processes sit idle. At this time, the parent process may decide to terminate some children based on its *MaxSpareServers* directive.

The Thundering Herd

The function of the 'accept mutex' (as this inter-process lock is called) is to keep request reception moving along in an orderly fashion. If the lock is absent, the server may show what's called the Thundering Herd syndrome.

---

[8] http://www.php.net/

Consider a football team[9] poised on the line of scrimmage. If the football players were Apacher processes all team members would go for the ball simultaneously at the snap. One process would get it, and all the others would have to lumber back to the line for the next snap.

Moving this much information around is obviously a lot of work, and a smart web server tries to avoid it whenever possible. Hence the revolving door construction. In recent years, many operating systems, including Linux and Solaris, have put code in place to prevent the Thundering Herd syndrome. Apache recognizes this and if you run with just one network listener, meaning one virtual host or just the main server, it will refrain from using an accept mutex. If you run with multiple listeners (for instance because you have a virtual host serving SSL requests), it will activate the accept mutex to avoid internal conflicts.

You can manipulate the accept mutex with the *AcceptMutex* directive. Besides turning the accept mutex off, you can select the locking mechanism. Common locking mechanisms include *fcntl*, *System V Semaphores* and *pthread locking*. Not all are available on every platform, and their availability also depends on compile-time settings. The various locking mechanisms may place specific demands on system resources: manipulate them with care.

Be careful of turning the accept mutex off: Apache automatically recognizes the single listener situation described above and knows if it is safe to run without mutex on your platform[10].

Tuning Tips

People often look for the 'magic tune-up' that will make their system perform four times as fast by tweaking just one little setting. The truth is, present-day UNIX derivatives are pretty well-adjusted out of the box and there is not a lot that needs to be done to make them perform optimally. However, there are a few things that an administrator can do to improve performance.

RAM and Swap Space

The mantra about RAM is: *more is better*. As discussed above, unused RAM is put to good use as file system cache. The Apache processes get bigger if you load more modules, especially if you use modules that generate dynamic content in-process like PHP and mod_perl. A large configuration file (many virtual hosts) also tends to inflate the process footprint. Finally, Apache 2.0 processes tend to have larger footprints than those of Apache 1.3.

While the various platforms treat their virtual memory in different ways, it is never a good idea to run with less disk-based swap space than RAM. The virtual memory system

---

[9] Any resemblance to the college in West Virginia is purely coincidental.

[10] To find out, run httpd with the –V flag and see if *–D SINGLE_LISTEN_UNSERIALIZED_ACCEPT* appears in the output. You can see which locking mechanisms are available by running httpd –L, and browsing the output for the AcceptMutex directive.

is designed to provide a fallback for RAM, but when you don't have disk space available and run out of swappable memory, your machine grinds to a halt. This can crash your box, requiring a physical reboot for which your hosting facility may smugly charge you.

Also, such an outage naturally occurs when you least want it: when the world has found your website and is beating a path to your door. If you have enough disk-based swap space available and the machine gets overloaded (remember, you still have MaxClients to keep things in hand), it may get very, very slow as the system needs to swap memory pages to disk and back, but when the load decreases the system should recover. Plan for available hard drive swap space of atleast twice your amount of RAM, perhaps up to four times in situations with frequent peaking loads. Remember to adjust this configuration whenever you upgrade RAM on your system. In a pinch, you can use a regular file as swap space: see the manual pages for the *mkswap* and *swapon* or *swap* programs.

ulimit: Files and Processes

Given a machine with plenty of RAM and processor capacity, you can run hundreds of Apache processes if necessary… and if your kernel will allow it. The Linux 2.2 kernel series by default limited the number of processes a user can run to 256, which is a fairly small number in computational circles.

Consider a situation in which several hundred web servers are running; if some of these need to spawn CGI processes, the maximum number of processes would occur quickly.

However, you can change this limit with the command *ulimit –u [newvalue]*. This must be changed before starting the server, since the new value will only be available to the current shell and programs started from it. In newer Linux kernels the default has been raised to 2048. There does not seem to be a process limit on Solaris. On FreeBSD, the number seems to be the rather unusual513. Since the default user shell on this system is *csh* and *ulimit* is a built-in command in Bourne-like shells, you need to start a Bourne shell or *bash* to see and set this value, and then start the web server from this shell.

Similarly, the kernel may limit the number of open files per process. This is generally not a problem for pre-forked servers, which just handle one request at a time per process. Threaded servers, however, serve many requests per process and much more easily run out of available file descriptors. You can increase the maximum number of open files per process by running the *ulimit –n [newvalue]* command.

Setting User Limits on Linux System Startup

You can set the *ulimit* parameters on bootup by editing the */etc/security/limits.conf* file. This file allows you to set soft and hard limits on a per-user or per-group basis; the file contains commentary explaining the options. To enable this, make sure that the file */etc/pam.d/login* contains the line

session required /lib/security/pam_limits.so

All items can have a 'soft' and a 'hard' limit: the first is the default setting and the second the maximum value for that item.

Solaris does not seem to have a similar mechanism for manipulating limit values at boot time: you will have to set them in your startup script(s).

Turn Off Unused Services and Modules

Many UNIX and Linux distributions come with a slew of services turned on by default. You probably need few of them: your web server does not need to be running sendmail, nor is it likely to be an NFS server, etc. Turn them off.

On Red Hat Linux, the *chkconfig* tool will help you do this from the command line. On Solaris systems, my approach is to inspect the */etc/rc[123].d* directories and to change the first character of the name of startup scripts I don't want to start automatically from S to s. Since the Solaris file system is case sensitive, this disables services without actually altering them into unrecognizability.

So, *S88sendmail* becomes *s88sendmail*. This way, the init process will pass them over but it's still evident to other sysadmins that they were once active. Remember that, while Solaris transitions through subsequent run levels on startup, the Linux initialization just executes all the scripts in the default run level directory. The default run level for a Linux web server should be 3: you don't need to run an X-Windows desktop on a web server so level 5 should not be necessary.

In a similar fashion, cast a critical eye on the Apache modules you load. Most binary distributions of Apache, and pre-installed versions that come with Linux distributions, have their modules enabled through the *LoadModule* directive.

A notable exception is the Apache *httpd* on Cobalt Raq servers, which has mod_perl compiled statically to run the GUI – despite the fact that the GUI Apache is running as an entirely different process from the one doing the actual serving. You cannot disable this instance of mod_perl. Other modules, however, may be culled: if you don't use their functionality and configuration directives, you can turn them off by commenting out the corresponding *LoadModule* lines. Read the documentation[11] on each module's functionality before deciding whether to keep it enabled. While the performance overhead of an unused module is small, it's also unnecessary.

Spread the Load

There are many ways to spread system load around. If you have more than one disk in the system, put the log files on a different disk from the static content. This means one disk will do mostly writing, the other mostly seeking and reading. Also, distribute your swap partitions across multiple disks.

If you have the luxury of multiple available machines, run your database on a different box than your web server. Another way to optimize load distribution is to generate your dynamic content on yet another machine: this is easy if your application server does not run in the web server process: Tomcat is more easily moved to a different machine than mod_perl or PHP.

In the latter two cases, you may be able to work with a machine that runs *httpd* to serve the static content, and forward the requests for dynamic content through to another

[11] http://httpd.apache.org/docs-2.0/ for Apache 2.0, and http://httpd.apache.org/docs/ for Apache 1.3

machine running also Apache, but with mod_perl or PHP. This machine should not need to serve any static content.

This is especially the case for Tomcat: have your Java programmer(s) make it easy to separate static content out of the webapp. Another approach would be to just serve images from a separate host. The problem with this is that your web site might become harder to test for functionality, since that functionality will depend on the infrastructure that hosts it.

Further Reading

- Ryan B. Bloom, *Apache Server 2.0: The Complete Reference*, 2002 McGraw Hill Osborne; ISBN 0-07-222344-8
- Patrick Killelea, *Web Performance Tuning, 2nd Edition*, 2002 O'Reilly & Associates; ISBN 0-596-00172-X
- http://httpd.apache.org/docs-2.0/misc/perf-tuning.html
- http://httpd.apache.org/docs/misc/perf-tuning.html
- Adrian Cockcroft, Richard Pettit and Sun Microsystems Press, *Sun Performance and Tuning: Java and the Internet (2nd Edition)*, 1998 Prentice Hall PTR; ISBN 0130952494

This presenation has ended.
Click anywhere to return.

_

